UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

Worksheet 9

Student Name:Suyash **UID:25MCI10054**
Branch:MCA (AI&ML) **Section/Group:MAM-1 A**
Semester:2nd **Date of Performance:07/04/2026**
Subject Name:- Technical Training -I **Subject Code: 25CAP-652**

Aim

To implement database triggers in PostgreSQL to automatically calculate values and enforce constraints during data insertion operations.

Tools Used

- PostgreSQL
- pgAdmin
- Windows Operating System

Objectives

- **Trigger Implementation**: To understand how to create and use triggers in PostgreSQL.
- **Automation**: To automate calculation of total payable amount using triggers.
- **Data Integrity**: To enforce constraints using trigger conditions.
- **Real-time Processing**: To execute logic automatically before inserting data.

Theory

Triggers are special database objects that automatically execute a function when a specified event (INSERT, UPDATE, DELETE) occurs on a table. In PostgreSQL, triggers are associated with trigger functions written in PL/pgSQL. They help maintain data integrity, enforce business rules, and automate tasks.

Experiment Steps:

Step 1: Creating Table

Query:

```
CREATE TABLE employee (
   emp_id INT PRIMARY KEY,
   emp_name VARCHAR(50),
   working_hours INT,
   perhour_salary NUMERIC,
   total_payable_amount NUMERIC
);
```

Step 2: Creating Trigger Function

Query:

```
CREATE OR REPLACE FUNCTION CACULATE_PAYABLE_AMOUNT()
RETURNS TRIGGER
AS
$$
BEGIN
   NEW.total_payable_amount := NEW.perhour_salary * NEW.working_hours;
   IF NEW.total_payable_amount > 25000 THEN
      RAISE EXCEPTION 'INVALID ENTRY BEACUSE PAYABLE AMOUNT CAN
NOT BE GREATER THAN 25000';
   END IF;
```

```
  RETURN NEW;
END;
$$ LANGUAGE PLPGSQL;
```

Step 3: Creating Trigger

Query:

```
CREATE OR REPLACE TRIGGER
AUTOMATED_PAYABLE_AMOUNT_CALCULATION
BEFORE INSERT
ON employee
FOR EACH ROW
EXECUTE FUNCTION CACULATE_PAYABLE_AMOUNT();
```

Step 4: Inserting Valid Data

Query:

```
INSERT INTO EMPLOYEE(emp_id, emp_name, working_hours, perhour_salary)
VALUES (1, 'AKASH', 10, 1000);
```

Output:

Data Output | Messages | Notifications

```
INSERT 0 1

Query returned successfully in 1 secs 45 msec.
```

Step 5: Inserting Invalid Data (Exception Case)

Query:

INSERT INTO employee(emp_id, emp_name, working_hours, perhour_salary)

VALUES (2, 'Ankush', 8, 100000);

Output:

Data Output Messages Notifications

```
ERROR:  INVALID ENTRY BEACUSE PAYABLE AMOUNT CAN NOT BE GREATER THAN 25000
CONTEXT:  PL/pgSQL function caculate_payable_amount() line 6 at RAISE

SQL state: P0001
```

Step 6: Retrieving Data

Query:

SELECT * FROM EMPLOYEE;

Output:

Data Output Messages Notifications

Showing rows: 1 to 1

	emp_id [PK] integer	emp_name character varying (50)	working_hours integer	perhour_salary numeric	total_payable_amount numeric
1	1	AKASH	10	1000	10000

Learning Outcome

- Students will be able to create and use triggers in PostgreSQL.
- Students will understand how to automate calculations using triggers.
- Students will enforce constraints using trigger conditions.
- Students will understand real-time execution of database logic.